

Coding & Solution

Date	10 JUNE 2026
Team ID	XXXXXX
Project Name	Voice-Based Concept Understanding Analyser (VBCUA)
Maximum Marks	5 Marks

Coding & Solution

This section evaluates the quality and completeness of your implemented code and the overall solution. Your submission should demonstrate working functionality, clean code practices, and alignment with your defined requirements.

Instructions:

1. Ensure your code is well-structured, commented, and follows best practices for your chosen technology stack.
2. The solution should address the core problem statement and implement the key functional requirements.
3. Include all source files, configuration files, and setup instructions needed to run the application.
4. Attach or link to your code repository (e.g., GitHub) in the table below.

Solution Summary

Field	Details
Repository Link / URL	https://github.com/soumaykohli/Voice-Based-Concept-Understanding-Analyser.git

Programming Language(s)	Python
Framework(s) Used	Streamlit
Key Features Implemented	* In-Browser Audio Recording & Media Upload Interface using dedicated Streamlit widgets. * Speech-to-Text Transcription using integrated OpenAI Whisper engine parameters. * Semantic Similarity Modeling powered by SentenceBERT context embeddings. * Speech Delivery Flaw Tracking evaluating silence bounds and tracking common verbal filler densities via Librosa signal pipelines. * Automated Performance Summary PDF Builder using the ReportLab document tool.
Pending / Incomplete Features	None. All primary milestones outlined in core project constraints are fully operational and verified

Setup / Run Instructions	System Prerequisites: Ensure Python 3.9+ and systemlevel audio dependencies (like <code>ffmpeg</code>) are configured. Install Environment Dependencies: Run <code>pip install -r requirements.txt</code> to gather necessary dependencies (<code>streamlit</code>, <code>librosa</code>, <code>torch</code>, <code>sentence-transformers</code>, <code>openai-whisper</code>, <code>reportlab</code>). Initialize the Web Server Backend: Fire up the live interactive portal interface environment terminal prompt by issuing the console instruction: <code>streamlit run app.py</code>



Code Quality Checklist

S.No	Criteria	Status (Yes / No)
1	Code is modular and organized into functions / classes	Yes
2	Meaningful variable and function names are used	Yes
3	Code includes comments / documentation where necessary	Yes

4	Error handling is implemented for critical operations	Yes
5	The application runs without critical errors	Yes
6	Code is committed to a version control repository	Yes

Additional Notes / Comments:

The architectural baseline features clean separation between processing layers. Heavy machinelearning tasks (Whisper, S-BERT) utilize caching decorators where appropriate to eliminate redundant weights initialization bottlenecks, ensuring fast local responsiveness.